

Chapter 7 Summary

Summary in Arabic

المخطط الذهني للفصل السابع

المتقدم SQL : الفصل 7

- 1. الاستعلامات المعقدة (Complex Queries)
 - 1.1 (Three-Valued Logic) والمنطق ثلاثي القيم NULL القيم
 - 1.2 (Nested Queries) الاستعلامات المتداخلة
 - (مرتبة vs بسيطة) أنواعها
 - (IN, ANY/SOME, ALL) مشغلات المقارنة
 - (Tuples) استخدام الأزواج المرتبة
 - 1.3 (Aliases) تجنب الغموض باستخدام أسماء مستعارة
- 2. القيود الدلالية والإجراءات التلقائية
 - 2.1 (القيود العامة) Assertions
 - 2.2 (المُحفّزات) Triggers
- 3. (الجدول الافتراضية) Views
 - تعريفها
 - فوائدها
 - قيود التعديل عليها
- 4. (Schema Modification) تعديل المخطط
 - إضافة/حذف/تعديل الجداول والحقول
 - التأثير على البيانات والقيود

⚠ **ملاحظة:** النص المرفق من الملف يركّز فقط على الجزء الأول من الفصل (حتى صفحة 13)، أي الاستعلامات المعقدة، ولا يحتوي على تفاصيل كافية عن الـ Views، Triggers، أو Schema Modification. لذا، سيركّز التلخيص على ما ورد في الملف، مع الإشارة إلى باقي المواضيع كعناوين عامة فقط.

التلخيص التفصيلي

1. الاستعلامات المعقدة (More Complex SQL Retrieval Queries)

تتيح لغة SQL ميزات متقدمة لاسترجاع البيانات بشكل أكثر تعقيدًا ودقة، مثل:

- الاستعلامات المتداخلة (Nested Queries)
- الـ Joins (بما في ذلك الـ Outer Joins)
- الدوال التجميعية (Aggregate Functions) مثل COUNT , SUM , AVG
- التجميع (GROUP BY)
- المعالجة الخاصة للقيم NULL

1.1 القيم NULL والمنطق ثلاثي القيم (Three-Valued Logic)

ما معنى NULL؟

القيمة NULL في قواعد البيانات لا تعني "صفر" أو "سلسلة فارغة"، بل تشير إلى:

- قيمة غير معروفة (Unknown)
- قيمة غير متاحة أو ممنوعة من العرض (Unavailable/Withheld)
- سمة غير منطبقة على السجل (Not Applicable)

خصائص الـ NULL:

- كل NULL يُعتبر مختلفًا عن أي NULL آخر (لا يمكن مقارنتها كقيم متساوية).
- مقارنة NULL = NULL تُعطي نتيجة UNKNOWN ، وليس TRUE أو FALSE .

المنطق ثلاثي القيم (3VL):

لا يستخدم منطقًا ثنائيًا (صحيح/خاطئ) فقط، بل ثلاثي القيم SQL:

- TRUE
- FALSE
- UNKNOWN تظهر عند مقارنة أي قيمة بـ ← UNKNOWN

مثال: ✓

```
SELECT * FROM EMPLOYEE WHERE Salary = NULL
```

هذه الجملة لا تُرجع أي سجل، لأن Salary = NULL تُقيّم كـ UNKNOWN ، والـ WHERE يُرجع فقط ما يُقيّم كـ TRUE .

كيفية التحقق من وجود NULL:

نستخدم:

- للتحقق من أن القيمة فارغة ← IS NULL
- للتحقق من أن القيمة غير فارغة ← IS NOT NULL

مثال صحيح: ✓

```
;SELECT Fname, Lname FROM EMPLOYEE WHERE Salary IS NULL
```

1.2 الاستعلامات المتداخلة (Nested Queries)

ما هي؟

عبارة SELECT-FROM-WHERE كاملة داخل جملة WHERE لاستعلام خارجي.

- الاستعلام الخارجي (Outer Query): يحتوي على الاستعلام المتداخل.
- الاستعلام الداخلي (Subquery): يُنفَّذ أولاً (في الحالة غير المرتبطة) ويُعيد مجموعة قيم.

مشغل IN :

يُستخدم لمقارنة قيمة واحدة مع مجموعة قيم تُرجعها الاستعلام الداخلي.

✓ مثال:

ابحث عن أسماء الموظفين الذين لديهم معالين (Dependents):

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Ssn IN (SELECT Essn FROM DEPENDENT)
```

مشغلات ALL و ANY / SOME :

- = ANY(...) ← يعادل IN(...)
- > ANY(...) ← صحيح إذا كانت القيمة أكبر من أي قيمة في المجموعة (أي أكبر من الأصغر على الأقل)
- > ALL(...) ← صحيح فقط إذا كانت القيمة أكبر من كل القيم في المجموعة (أي أكبر من الأكبر)

✓ مثال:

ابحث عن الموظفين الذين راتبهم أكبر من راتب أي موظف في قسم 5:

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Salary > ANY (SELECT Salary FROM EMPLOYEE WHERE Dno = 5)
```

✓ مثال آخر:

ابحث عن الموظفين الذين راتبهم أكبر من راتب كل موظف في قسم 5:

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5)
```

استخدام الأزواج المرتبة (Tuples):

يمكن مقارنة أكثر من عمود في آنٍ واحد باستخدام الأقواس.

مثال: ✓

ابحث عن الموظفين الذين لديهم معال يحمل نفس الاسم الأول وجنسه:

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
) WHERE (E.Fname, E.Sex) IN
SELECT D.Dependent_name, D.Sex
FROM DEPENDENT AS D
WHERE D.Essn = E.Ssn
;
```

1.3 الاستعلامات المتداخلة المرتبطة (Correlated Nested Queries)

- في هذا النوع، يعتمد الاستعلام الداخلي على قيمة من الاستعلام الخارجي.
- يُنفَّذ مرة واحدة لكل سجل في الاستعلام الخارجي.
- لا يمكن "طيّه" (collapse) إلى JOIN بسهولة دائمًا، لكن أحيانًا يمكن.

مثال من الملف (Q16): ✓

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
) WHERE E.Ssn IN
SELECT D.Essn
FROM DEPENDENT AS D
WHERE D.Essn = E.Ssn
AND E.Fname = D.Dependent_name
AND E.Sex = D.Sex
;
```

الصيغة المكافئة باستخدام (Q16A) JOIN: ✓

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn
AND E.Sex = D.Sex
; AND E.Fname = D.Dependent_name
```

💡 ملاحظة: استخدام AS لإنشاء أسماء مستعارة (Aliases) مثل E و D يُحسِّن الوضوح ويقلل الغموض، خاصة عند وجود جداول متكررة أو مراجع ذاتية.

1.4 تجنب الغموض باستخدام الأسماء المستعارة (Aliases)

- عند الرجوع إلى نفس الجدول أكثر من مرة (مثل في Self-Join أو Subquery مرتبطة)، يجب استخدام أسماء مستعارة.
- يُوصى باستخدام AS صراحة لتحسين قابلية القراءة.

✓ مثال جيد:

```
SELECT E1.Fname, E1.Lname
FROM EMPLOYEE AS E1, EMPLOYEE AS E2
WHERE E1.Super_ssn = E2.Ssn
; AND E2.Fname = 'James'
```

ملخص النقاط الأساسية

المفهوم	التوضيح
NULL	لا تعني "فارغة" بل "غير معروفة/غير منطبقة"، ولا تساوي أي شيء (حتى نفسها)
IS NULL	الطريقة الصحيحة للتحقق من القيم الفارغة
IN	يُستخدم للمقارنة مع مجموعة قيم
ANY / SOME	يكفي أن تتحقق المقارنة مع أحد العناصر
ALL	يجب أن تتحقق المقارنة مع جميع العناصر
الاستعلام المرتبط	يعتمد على متغير من الاستعلام الخارجي، ويُنفذ لكل سجل خارجي
الأسماء المستعارة	ضرورية لتجنب الغموض وتحسين الأداء والقراءة

➔ ما لم يُذكر في الملف (لكن هو جزء من الفصل) SOON

رغم أن الملف المرفق ينتهي عند صفحة 13 (وهو يركّز فقط على الاستعلامات المعقدة)، فإن باقي الفصل يتناول:

2. القيود الدلالية والإجراءات التلقائية

- **Assertions:** قيود عامة تُفرض على قاعدة البيانات (مثل: "إجمالي الرواتب في القسم لا يتجاوز مليون").
- **Triggers:** إجراءات تُنفذ تلقائيًا عند حدوث أحداث (INSERT, UPDATE, DELETE).

3. الـ Views

- جداول افتراضية لا تخزن بيانات، بل تُعرّف كاستعلام.
- تُستخدم للأمان، التبسيط، وتوفير واجهات مستخدم مختلفة.

4. تعديل المخطط (Schema Modification)

• أوامر مثل:

- ALTER TABLE ... ADD COLUMN
- ALTER TABLE ... DROP COLUMN
- DROP TABLE

• قد تؤثر على القيود، الفهارس، والبيانات الموجودة.

⚠ هذه المواضيع لم تُذكر تفاصيلها في الملف المرفق، لذا لم نتعمق فيها.

خاتمة ✓

هذا التلخيص غطى جميع المفاهيم الواردة في الملف المرفق بدقة، مع:

- ترتيب منطقي حسب مخطط ذهني
- أمثلة عملية
- توضيح الفروق الدقيقة (مثل NULL vs 0، ANY vs ALL)
- نصائح لكتابة استعلامات واضحة وصحيحة

Summary in English

🧠 Mind Map of Chapter 7

Chapter 7: More SQL – Complex Queries, Triggers, Views, and Schema Modification

- |
- ├─ 1. More Complex SQL Retrieval Queries
 - | ── 1.1 NULL Values and Three-Valued Logic
 - | ── 1.2 Nested Queries
 - | ── IN operator
 - | ── ANY/SOME and ALL operators
 - | ── Tuple comparisons (multi-attribute)
 - | ── 1.3 Correlated Nested Queries & Use of Aliases
- ├─ 2. Specifying Semantic Constraints as Assertions and Actions as Triggers
(Mentioned in outline only – not detailed in provided slides)
- ├─ 3. Views (Virtual Tables) in SQL
(Mentioned in outline only – not detailed in provided slides)
- |

4. Schema Modification in SQL

(Mentioned in outline only – not detailed in provided slides)

⚠ **Note:** The provided PDF excerpt (Slides 7-2 to 7-13) covers **only Section 1** in detail: **More Complex SQL Retrieval Queries**. The other three topics (Assertions/Triggers, Views, Schema Modification) appear only in the chapter outline and are **not elaborated** in the given material. Therefore, this summary focuses exclusively on the content actually presented.

Detailed Summary (Based on Provided Slides)

1. More Complex SQL Retrieval Queries

SQL provides advanced features to express complex data retrieval needs, including:

- **Nested queries**
- **Joined tables and outer joins** (in the `FROM` clause)
- **Aggregate functions** (e.g., `COUNT`, `SUM`, `AVG`)
- **Grouping** (`GROUP BY`)
- Special handling of `NULL` values

1.1 Comparisons Involving `NULL` and Three-Valued Logic

What does `NULL` mean?

In SQL, `NULL` represents:

- An **unknown** value
- An **unavailable or withheld** value
- An attribute that is **not applicable** in a given context

Key properties of `NULL` :

- Each `NULL` is considered **distinct from every other** `NULL` .
- The comparison `NULL = NULL` is **not valid** and **does not evaluate to** `TRUE` .

Three-Valued Logic (3VL):

SQL uses **three truth values** in logical expressions:

- `TRUE`
- `FALSE`

- UNKNOWN (equivalent to “maybe”)

✅ Example:

The condition `Salary = NULL` evaluates to UNKNOWN, **not** TRUE or FALSE.

Therefore, it **will not select any rows** in a WHERE clause, because WHERE only returns rows where the condition is TRUE.

Checking for NULL :

Use the special predicates:

- IS NULL
- IS NOT NULL

✅ Correct usage:

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Salary IS NULL;
```

1.2 Nested Queries, Tuples, and Set Comparisons

What is a nested query?

A **complete** SELECT-FROM-WHERE block embedded inside the WHERE clause of another query.

- **Outer query:** the main query
- **Subquery (nested query):** the inner query

The IN Operator:

Used to test whether a value matches **any value** in a set returned by a subquery.

✅ Example:

Find names of employees who have dependents:

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Ssn IN (SELECT Essn FROM DEPENDENT);
```

Tuple Comparisons:

You can compare **multiple attributes at once** by using **parenthesized tuples**.

✓ Example:

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE (E.Fname, E.Sex) IN (
    SELECT D.Dependent_name, D.Sex
    FROM DEPENDENT AS D
    WHERE D.Essn = E.Ssn
);
```

ANY / SOME and ALL Operators:

- = ANY(...) is **equivalent to** IN(...)
- Other operators can be combined with ANY / SOME or ALL :
 - > ANY(...) : true if greater than **at least one** value
 - > ALL(...) : true only if greater than **every** value in the set

✓ Examples:

```
-- Employees earning more than someone in Dept 5
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Salary > ANY (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);
```

```
-- Employees earning more than everyone in Dept 5
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);
```

1.3 Correlated Nested Queries and Aliases

Correlated Nested Query:

A subquery that **references attributes from the outer query**.
It is **evaluated once for each tuple** in the outer query.

✓ Example (from Slide 7-12):

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN (
```

```

SELECT D.Essn
FROM DEPENDENT AS D
WHERE D.Essn = E.Ssn
      AND E.Fname = D.Dependent_name
      AND E.Sex = D.Sex
);

```

Equivalent Non-Nested (Joined) Form:

Such queries can often be rewritten using **explicit joins**:

✅ Equivalent query (Q16A):

```

SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn
      AND E.Sex = D.Sex
      AND E.Fname = D.Dependent_name;

```

Importance of Aliases:

- Use **table aliases** (e.g., AS E , AS D) to:
 - Avoid ambiguity
 - Improve readability
 - Enable self-joins or multiple references to the same table

💡 **Best Practice:** Always define explicit aliases for tables in complex queries.

📌 Key Concepts Recap

Concept	Explanation
NULL	Represents unknown, unavailable, or inapplicable data; not equal to anything (including itself)
Three-Valued Logic	Logical results can be TRUE , FALSE , or UNKNOWN
IS NULL	Only correct way to test for null values
IN	Checks membership in a set returned by a subquery
ANY / SOME	Satisfies condition if at least one comparison is true
ALL	Satisfies condition only if all comparisons are true
Tuple Comparison	Compare multiple columns simultaneously using (col1, col2) syntax

Concept	Explanation
Correlated Subquery	Inner query depends on outer query; executed per outer row
Aliases (AS)	Essential for clarity and correctness in complex queries

Topics Mentioned But Not Covered in Provided Slides

Although listed in the chapter outline, the following sections **do not appear in the provided slides (7-2 to 7-13)**:

- **Assertions:** General constraints to enforce semantic rules (e.g., “Total salary per department $\leq$ \$1M”)
- **Triggers:** Procedures that automatically execute in response to events (INSERT/UPDATE/DELETE)
- **Views:** Virtual tables defined by queries, used for security, simplification, and abstraction
- **Schema Modification:** Using ALTER TABLE , DROP TABLE , etc., to change database structure

These topics are part of Chapter 7 in the full textbook but are **outside the scope** of the uploaded excerpt.

Conclusion

This summary faithfully reflects **all content from the provided Chapter07.pdf slides**, organized logically with:

- A clear mind map
- Conceptual explanations
- Practical SQL examples
- Important notes and best practices

It ensures **no loss of information** while enhancing clarity and study efficiency.